

APPENDIX B:**Declaration of Original Work for SC2002 Assignment**

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Course	Lab Group	Signature / Date
Tan Kai Xin	CE	SCE2	Kp 22/11/2025
Abel Koh Chung kiat	CE	SCE2	MR 22/11/25
Wong Jun Kei, Dylan	CE	SCE2	WJ 22/11/25
Ong Yi Sheng	CE	SCE2	CS 22/11/25
Ryan Tan Rui Zhe	CE	SCE2	RZ 22/11/25

Important notes:

1. Name must **EXACTLY MATCH** the one printed on your Matriculation Card.
2. Student Code of Academic Conduct includes the latest guidelines on usage of Generative AI and any other guidelines as released by NTU.

1. Design Considerations & Analysis

a) Problem and Requirements

The internship placement management system we designed based on the project assignment serves as a central hub for Students, Company Representatives and Career Center Staff to manage internships. This includes making and approving internships, applying for and accepting internships. We applied Object-Oriented Design (OOD) principles like encapsulation, scalability and extensibility, as well as good design principles such as SOLID to help us make a better project. As indicated, the project is also made on the Command Line Interface.

b) Required Implemented Features

• Login & Registration System:

- Allows Students, Company Representatives and Career Center Staff to sign in.
- Authentication based on credentials are saved in CSV files.
- Company representatives must be approved by Career Center Staff before they can log in

• Individual User Systems

Students	• View internships based on internship levels, major, and visibility status • Apply for up to 3 internships at a time and track their application status (PENDING, SUCCESSFUL, UNSUCCESSFUL) • If sign up is unsuccessful, allow for registration • Accept up to 1 internship application • Request to withdraw internship application, or from an internship role
Company Representatives	• Create up to 5 internship opportunities • Approve or reject student applications • Toggle internship visibility • View applications and manage slots based on successful application
Career Center Staff	• Approving company representatives, internships, and withdrawal requests • Generate custom reports based on filters, including internship status, preferred major, and company name

c) Additional Implemented Features

- Generate Quick Report

- This will create a custom report instantly, without any filters
- 2 ways to export the report
 - Can export the report as a .txt. file. This allows Career Center Staff to be able to Generate quick report (a report without filters)
 - Can display the report on the terminal UI instead.
- Error checking for the code
 - Since User Input is always prone to errors, we added error checking in places such as dates, Internship level, Application Statuses.
- After closing the programme, all user data is saved into CSV files, so that users do not need to sign up again.

2. System Architecture & Design

a) Entity-Boundary-Control Design Principle

● Approach

Our design followed the Entity-Boundary-Control (EBC) architecture. This ensures that the project's software codes have a clean separation between the user interface (Boundary), the business logic (Control), and the data (Entity). Utilising this design also ensures that each part of the system has a single responsibility and can evolve independently of others, thus adhering to the Object-Oriented Single Responsibility Principle and Dependency Inversion Principle.

● Entity Classes

Entity classes manage the core data and business logic of the system. The Key entities are:

- **User** (abstract class): Contains common properties for all users (`id`, `name`, `password`), with methods to authenticate and change passwords. Below are the attributes for this class:

```
public abstract class User { 7 usages 3 inh
    private String userID; 4 usages
    private String name; 4 usages
    private String password; 4 usages
```

- **Student** (extends `User`): Contains `yearOfStudy`, `major`, and a list of up to 3 internship applications. It includes logic for applying for internships, viewing applications, and accepting offers. Below are the attributes for this class

```
public class Student extends User {
    private int yearOfStudy; 3 usages
    private String major; 3 usages
    private String studentID; 1 usage
    private Internship internship; 9 usages
    private Application[] applications; 24 usages
    private String email; 3 usages
```

- **CompanyRepresentative** (extends `User`): Contains `companyName`, `department`, and a list of internships. Company representatives can create internships

and approve or reject student applications. Below are the attributes for this class

```
public class CareerCenterStaff extends User{ 34 usages
    private String staffDepartment; 3 usages
    private String email; 3 usages
    private String careerCenterStaffID; 1 usage
```

- **CareerCenterStaff** (extends User): Contains staffDepartment and handles approval processes for representatives, internships, and withdrawal requests. Below are the attributes
- **Internship**: Models internship opportunities, with attributes such as title, level, companyName, status, and a list of applications.
- **Application**: Models a student's application to an internship, with attributes like status, applicationDate, and references to Student and Internship. Below shows the attributes for this class.

```
public Application(String ID, Internship opportunity, Student applicant){
    applicationDate = new Date();
    internship = opportunity;
    student = applicant;
    applicationID = ID;
    status = "Pending";
}
```

These classes interconnect with each other through association relations. For example, a Student can have many Applications, and Internship can have many Applications

- **Control Classes**

The Control classes contain the system's workflow logic. The control classes include:

- **Main**: Manages the flow of the application that consists of user login, navigation between menus, and interaction with controllers.
- **AuthenticationController**: Handles authentication and ensures users can log in with the correct credentials.
- **StudentController**: Manages student actions, including applying for internships, accepting offers, and viewing applications.
- **CompanyRepController**: Handles company representative actions such as creating internships, approving/rejecting applications, and toggling visibility.
- **StaffController**: Manages career center staff actions like approving company representatives, internships, and processing withdrawal requests.
- **RegistrationController**: Manages registration of users such as creation of their user ID
- **ProfileController**: Manages user profile operations, allowing users to change their passwords and view their profile details. It handles different types of users by

displaying specific information based on whether the user is a Student, Company Representative, or Career Center Staff.

- **InternshipBrowsingController**: Controls the browsing and filtering of internships in a system, allowing users to configure various filters like status, major, and company. It also handles sorting and displays the filtered internship **listings to users based on their criteria and permissions**
- **FileController**: This handles file input/output operations for user data, providing methods to load and save students, staff, and company representatives from/to CSV files. These control classes handle and coordinate the logical implementation of the different actions available to the User class and all its subclasses like CompanyRepresentative, CareerCenterStaff and Student. Below shows the registerCompanyRep method present in the RegistrationController.

```
public CompanyRepresentative registerCompanyRep(String[] details) { 1 usage
    String email = details[0];
    String name = details[1];
    String companyName = details[2];
    String department = details[3];
    String position = details[4];

    String companyRepID = generateNextCompanyRepID();

    CompanyRepresentative newRep = new CompanyRepresentative(
        companyRepID, name, password: "password", companyName, department
    );

    companyRepresentatives.add(newRep);

    return newRep;
}
```

- **Boundary Classes**

The Boundary Classes are responsible for handling user input and output. The main boundary classes include:

- **LoginBoundary**: Responsible for prompting the user for credentials and handling login logic. Below shows how we get the password and username.

```
public String getUsername() { 1 usage
    System.out.print("Enter your username: ");
    return scanner.nextLine();
}

public String getPassword() {
    System.out.print("Enter your password: ");
    return scanner.nextLine();
}
```

- **RegistrationBoundary**: Allows for the registration of new users (students, company representatives, and staff). Below shows the code for successful registration.

```
// display registration success message
public void displayRegistrationSuccess(String userType) { 3 usages
    System.out.println("You have registered! Default password is 'password'.");
    if (userType.equals("CompanyRep")) {
        System.out.println("Please wait for approval from Career Center Staff.");
    }
}
```

- **MenuBoundary**: Displays the main menu and role-specific options for users. Below shows the method for displaying our main menu.

The 3 boundary classes act as independent command-line interface (CLI) screens. They do not call each other directly. Instead, the Main and Controllers class coordinate when each boundary class is used. This keeps the UI modular and loosely coupled to the business logic.

```
// display main menu and get choice
public int displayMainMenu() { 1 usage
    System.out.println("\n=== MAIN MENU ===");
    System.out.println("1. Profile Management");
    System.out.println("2. Role-specific Functions");
    System.out.println("3. Browse Internships");
    System.out.println("4. Logout");
    System.out.println("5. Switch off program");
    System.out.print("Enter your choice: ");
    return getMenuChoice();
}
```

b) Object-Oriented Design Principle (SOLID)

- **Single Responsibility Principle**

Classes within the system adhere to the Single Responsibility Principle as much as possible. Entity classes such as User, Student, CareerCentreStaff and CompanyRepresentative primarily hold data. As such their functions revolve around access, modification and displaying of the data they hold. Classes such as StudentController and other Controller classes help to perform the logic of the class specific functions at the system level during run time including calling the getter and setter functions of the other entity classes when necessary. These controller classes however do not store information, they leave that to the entity classes in the system. Furthermore, their attributes are only there as object references to allow them to access and modify data stored in the system at run time.

```
7 public class StudentController {
8     private final Scanner scanner;
9     private final List<Internship> internships;
```

As shown above, StudentController attributes are not for storing data, as they do not instantiate anything. They exist to allow the StudentController to use the system data and

the system scanner object instances to prevent bugs from occurring. Especially when multiple scanner instances are instantiated.

- **Open-Closed Principle**

The system is closed to modification but open to extension. This can be seen in the user classes, Since all entity classes inherit from a `User` abstract class, during runtime downcasting is used to ensure the system runs no matter the class type of the user. As long as it inherits from the `User` class type. This means the system can accommodate extensions when there is the addition of new `User` subclasses and types, as long as they inherit from the `User` subclass.

```
public abstract class User { 7 usages 3 inheritors
    private String userID; 4 usages
    private String name; 4 usages
    private String password; 4 usages
```

Parent Class – User Class

```
public class Student extends User {
    private int yearOfStudy; 3 usages
    private String major; 3 usages
    private String studentID; 1 usage
    private Internship internship; 9 usages
    private Application[] applications; 24 usages
    private String email; 3 usages
```

Children classes, they all “extends User”

```
public class CareerCenterStaff extends User { 34 usages
    private String staffDepartment; 3 usages
    private String email; 3 usages
    private String careerCenterStaffID; 1 usage
```

```
public class CompanyRepresentative extends User { 38 usages
    private String companyName; 5 usages
    private String department; 3 usages
    private String position; 3 usages
    private String status; 6 usages
    private List<Internship> internships; 7 usages
    private String email; 4 usages
    private static final int MAX_INTERNSHIPS = 5; 2 usages
```

`InternshipBrowsingController` class can accommodate the extension of the system when new user classes are added, since it takes in the parent class `User` that all user classes must inherit from instead of any specific subclass. This allows `InternshipBrowsingController` to accept any newly added classes during runtime without throwing up errors.

- **Liskov Substitution Principle**

The aforementioned usage of the abstract class `User` ensures that common functions that all user subclasses need to have, such as `changePassword`, is implemented. This prevents the subclasses from throwing up errors when the parent class does not, adhering to the Liskov Substitution Principle.

- **Interface Segregation Principle**

To meet deadline schedules, it was not possible to create interfaces for the system. The closest we have to implementing the Interface Segregation Principle is in the usage of different controller classes to ensure that users of different subclasses do not have access to functions that they will not apply or use. For example, `CompanyRepresentative` cannot create applications for an internship, which is a `Student` function.

- **Dependency Injection Principle**

The best application of this is in the usage of controller classes in the main system. Low-level Controller classes handle the logic of the system, allowing `Main`, a high-level class, to not have to implement the logic.

```

229 private static void handleCompanyRepFunctions() {
230     CompanyRepresentative rep = (CompanyRepresentative) currentUser;
231     boolean inMenu = true;
232
233     while (inMenu) {
234         int choice = menuBoundary.displayCompanyRepMenu();
235
236         switch (choice) {
237             case 1 -> CompanyRepController.createInternshipOpportunity(rep, scanner, internships);
238             case 2 -> CompanyRepController.viewCompanyApplications(rep, scanner);
239             case 3 -> CompanyRepController.approveRejectApplications(rep, scanner);
240             case 4 -> CompanyRepController.toggleInternshipVisibility(rep, scanner);
241             case 5 -> CompanyRepController.viewMyInternships(rep);
242             case 6 -> inMenu = false;
243             default -> menuBoundary.displayInvalidChoice();
244         }
245     }
246 }

```

In Main, the usage of controllers ensures that Main carries out the high-level task of controlling the flow of the program when a CompanyRepresentative is accessing the system. This leaves the CompanyRepController to handle the actual logic of the actions that the user wants the system to perform which is a low-level task.

c) Object-Oriented Principles

• Encapsulation and Information Hiding

All our entity classes have variables that have the “private” attribute, making sure that the variables can only be accessed using public getter and setter methods that are implemented in the same class. This ensures that we do not accidentally corrupt the data of any entity by accident.

```

public class CompanyRepresentative extends User { 39 usages
    private String companyName; 5 usages
    private String department; 3 usages
    private String position; 3 usages
    private String status; 6 usages
    private List<Internship> internships; 7 usages
    private String email; 4 usages
    private static final int MAX_INTERNSHIPS = 5; 2 usages

    public CompanyRepresentative(String globalID, String representativeName, String password,
        String companyName, String department, String position) {
        super(globalID, representativeName, password);
        this.companyName = companyName;
        this.department = department;
        this.position = position;
        this.status = "Pending";
        this.internships = new ArrayList<>();
        this.email = email;
    }

    public String getCompanyName() { return companyName; }

    public void setCompanyName(String companyName) { this.companyName = companyName; }

    public String getDepartment() { return department; }
}

```

CompanyRepresentative class' private attributes and public getter / setter methods

• Polymorphism

Polymorphism is used in the main interface. While the main interface is largely the same, there are certain parts that are specific to the ‘currentUser’ (either Student, CompanyRep or

Staff). Hence, when calling for these specific functions, `currentUser` is used, and whatever the `currentUser` is will override the function based on the `currentUser`.

```
private static void handleProfileManagement() { 1 usage
    int choice = menuBoundary.displayProfileMenu();

    switch (choice) {
        case 1 -> profileController.changePassword(currentUser);
        case 2 -> profileController.viewProfile(currentUser);
        default -> menuBoundary.displayInvalidChoice();
    }
}
```

handleRoleSpecificFunction shows Polymorphism

- **Abstraction**

Abstraction is when only essential behaviour is shown. This occurs when classes use the methods of other classes, without knowing how the method is implemented. In the example below, `changePassword` makes use of `user.changePassword`, thus implementing `changePassword` without knowing what type `User` is.

```
public boolean changePassword(User user) { 1 usage
    System.out.print("Enter current password: ");
    String currentPass = scanner.nextLine();

    if (currentPass.equals(user.getPassword())) {
        System.out.print("Enter new password: ");
        String newPass = scanner.nextLine();
        user.changePassword(newPass);
        System.out.println("Password changed successfully!");
        return true;
    } else {
        System.out.println("Current password is incorrect!");
        return false;
    }
}
```

Example of abstraction in changePassword

d) Tradeoffs

One of the biggest hurdles we had in the project was balancing the dateline of a working program against the long-term benefits of a more extensible architecture. We recognise that a well designed program would involve the use of interfaces to create a layer of abstraction, providing low coupling to make the program more maintainable and future-proof. This approach would adhere to the SOLID principles for a more robust structure. However, implementing this interfaced architecture would require a significant amount of time and effort to refactor our logic and rework the class dependencies. We ultimately prioritise a functional deliverable and development speed, opting for a simpler and more direct implementation which sacrifices long term extensibility for immediate completion.

3. Class Diagram

a) Class Diagram ([Full class diagram Photo](#))

b) Inter-class Relationships

Our class diagram showed several types of relationships to model the constraints of the Internship Management System mentioned in the project assignment:

- **Inheritance (Generalization):**

We implemented an inheritance hierarchy where `Student`, `CompanyRepresentative`, and `CareerCenterStaff` extend the abstract `User` class. This allows us to follow the **Liskov Substitution Principle**, as the `AuthenticationController` and `ProfileController` can operate on the generic `User` type without needing to know the specific subclass logic.

- **Composition (Strong Ownership)**

The line with solid diamond exists between `CompanyRepresentative` and `Internship`. This shows internship listing is strictly dependent on the existence of the company representative. If a representative account is removed, their created listings are logically removed as well. Same logic applies to `Internship` as it has a composition relationship with `Application`, as an application cannot exist without the internship it targets.

- **Aggregation (Weak Ownership)**

The line with hollow diamond between `Student` and `Application` reflects that while a student "owns" their applications, the student entity exists independently. If an application is deleted, the student remains valid.

- **Dependencies**

The Controller classes (Green package) have dash arrows with the Entity classes (Yellow package). This means that while Controllers manipulate Entity data, the Entities themselves remain passive and do not depend on the control logic, preventing circular dependencies.

c) Multiplicity & Constraints

We enforced business rules directly through the cardinality defined in the Class Diagram:

- **Application Limit (0..3):**

The relationship between `Student` and `Application` is defined as 1 to 0..3. This explicitly restricts a student from having more than three active applications at any given time.

- **Internship Creation Limit (0..5)**

The relationship between `CompanyRepresentative` and `Internship` is defined as 1 to 0..5. This constrains a single representative to managing a maximum of five internship listings simultaneously to ensure quality control

d) Helper Classes & Value Objects

To maintain the Single Responsibility Principle, we change filtering logic into a Value Object called `InternshipFilter`. Instead of putting search engines in the `Internship` entity, this class encapsulates the state of user search criteria (e.g., preferred major, status), which is then used by the `InternshipBrowsingController`.

4. UML Sequence Diagram

a) Sequence Diagram([Repository of sequence diagram Photos](#))

`CompanyRepresentative`, `Student`, `CareerCentreStaff`, `Main` sequence diagrams can be found in this Github link as they are quite big and cannot be shown here clearly.

b) Internship management ([Internship Management Scenario Photo](#))

A `CompanyRepresentative` chooses to create an internship which causes `Main` to send a message to `CompanyRepController` to create internships. A series of error checking and collection of user inputs is done by the controller to input the correct information into the corresponding fields of the internship constructor.

c) Application Process [Sequence Diagram of Application Process Scenario](#)

Assuming that a student has already made an internship application, the company representative can choose to view all applications. After that, the company representative can choose the option to approve/reject applications. This gets the `Main` to send a message to the `CompanyRepController` to get it to run the function `approveRejectApplications`. This function then runs the process of displaying the appropriate information and getting user inputs from the company representative to identify which `Student` application to approve or reject.

5. Reflection & Lessons Learned

a) Knowledge Learnt

- **Planning sufficiently before starting to code**

We realised that to make better code, more planning and thinking should be done beforehand to thoroughly understand the problem we are tackling. This will result in less major errors, and also less debugging, which can be very frustrating while working on the code. We started coding already after having ideas for the broadstrokes of the project, but we had yet to fully decide upon the minute details and edge cases, such as figuring out when exactly an internship should be “filled”.

- **Usage of EBC software architecture**

By applying the EBC software architecture to our program, we were able to clearly organise our code into individual classes that each had their own unique purpose and function. This makes modification of the code easier which is helpful in the debugging stage. Since the different classes and segments are separated, their function is independent of the logic implementation within the other classes. Thus, when debugging, pin down which class contains the bugged code

and when editing it, the EBC architecture minimises, if not, outright removes the need to edit the other classes that interact with the buggy class.

- **Usage of Object-Oriented Principles**

The usage of Object Oriented Principles helped to ensure that the code we wrote is flexible enough to handle expansion of the system. SOLID Principles such as Open Closed Principle and the Liskov Substitution Principle makes space for the addition of new user classes apart from the three that we have currently implemented. Furthermore, principles such as the Single Responsibility Principle makes our code easier to debug, modify and read as each of the classes used has a different specific purpose, meaning when bugs occur it is easier to narrow down which file should be accessed or modified depending on the error message that occurs.

- b) **Difficulties Encountered**

- **Working on the same code together**

Since we are all working on the same programme, we had to ensure that all the variables were decided beforehand, and we were all clear about what each variable did, which was time-consuming and frustrating at times. Even with all the planning, there were still some mistakes, but it did help make combining code easier.

- **Coding Consistency & Debugging**

Following the same naming conventions was a significant challenge. We often encountered bugs simply because of capitalization errors (e.g., `studentId` vs `StudentId`). Similarly, we had instances where one module checked for "Rejected" while another checked for "Unsuccessful." These mismatches didn't cause compilation errors, but it produced the wrong results, making debugging extremely time-consuming.

- c) **Improvements**

- **Coding Consistency & Debugging**

Currently, our input validation logic is scattered across various classes. A significant improvement would be creating a utility class (e.g. `InputHelper`) to handle all user inputs instead of manually writing error checks for every specific prompt.

- **Implement Interfaces for User Sub-Class Controllers**

Interfaces can be made for the controller classes to implement. This will help our code abide by the Interface Segregation Principle. The interfaces will enforce the need to implement common User functions while allowing additional functions to be added for the different specific subclasses as needed.

- **Abstraction of Data**

We should have implemented a dedicated Database class to act as a single source of truth for our data. Currently, data lists are passed between various controllers, which increases coupling and the risk of data inconsistency.

6. **GitHub Link**

https://github.com/rtrz02/SC2002_SCE2_GRP4.git